

EngSci 721

Inverse Problems and Learning From Data

Oliver Maclaren (oliver.maclaren@auckland.ac.nz)

Lecture 14: Neural models

Topics:

- An attempt to learn about neural networks via matrix calculus!

Module overview

Inverse Problems and Learning From Data (*Oliver Maclaren*)

[~14 lectures/3 tutorials]

1. *Basic concepts* [5 lectures + 1 Tutorial]

Forward vs inverse problems. Well-posed vs ill-posed problems. Algebra and calculus of inverse problems (left and right inverses, generalised and pseudo inverses, resolution operators, matrix calculus). Representing higher dimensional problems (image data etc).

2. *Instability and regularisation* [6 lectures + 1 Tutorial]

Instability and related issues for generalised inverses. Introduction to regularisation and trade-offs. Tikhonov regularisation. Higher-order Tikhonov regularisation. Sparsity and regularisation using different norms. Truncated singular value decomposition. Iterative regularisation, including stochastic/mini-batch gradient descent.

2. *Further topics* [3 lectures + 1 Tutorial]

Regularisation parameter choice, including statistical and machine learning views of regularisation. Confidence sets for linear and nonlinear models. Physics-informed machine learning and neural networks.

EngSci 721 : Lecture 14: Neural models

- An attempt to apply matrix calculus by hand to understand neural nets!

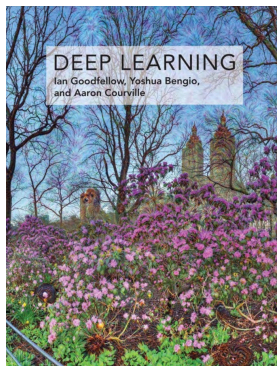
→ warning: surprisingly complex & unstandardised across lit

↳ done via AD libraries in practice
↳ Autograd (Pytorch, Jax etc)

see also eg:

Matrix Calculus (for Machine Learning and Beyond)

Lecturers: Alan Edelman and Steven G. Johnson
Notes by Paige Bright, Alan Edelman, and Steven G. Johnson
Based on MIT course 18.S096 (now 18.063) in IAP 2023



The Matrix Calculus You Need For Deep Learning

[Terence Parr](#) and [Jeremy Howard](#)

(Terence is a tech lead at Google and ex-Professor of computer/data science in [University of San Francisco's MS in Data Science program](#). You might know Terence as the creator of the ANTLR parser generator. For more material, see [Jeremy's latest course](#) and [University of San Francisco's Data Institute](#) ([in-person version of the deep learning course](#).)

Please send comments, suggestions, or fixes to [Terence](#).

[Printable version](#) (This HTML was generated from markup using [bookish](#)). A [Chinese version](#) is also available (content not verified by us).

Abstract

This paper is an attempt to explain all the matrix calculus you need in order to understand the training of deep neural networks. We assume no math knowledge beyond what you learned in calculus 1, and provide links to help you refresh the necessary math where needed. Note that you do **not** need to understand this material before you start learning to train and use deep learning in practice; rather, this material is for those who are already familiar with the basics of neural networks, and wish to deepen their understanding of the underlying math. Don't worry if you get stuck at some point along the way—just go back and reread the previous section, and try writing down and working through some examples. And if you're still stuck, we're happy to answer your questions in the [Theory category at forums.fast.ai](#). Note: There is a [reference section](#) at the end of the paper summarizing all the key matrix calculus rules and terminology discussed here.

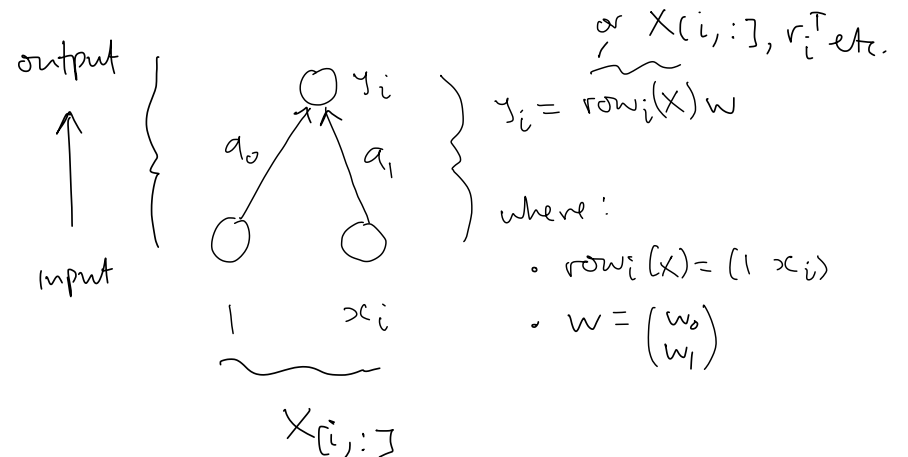
Linear models (again)

Consider

$y_i = w_0 + w_1 x_i$ for observations $i=1, \dots, m$

$$\text{ie: } \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

We can write as a 'network diagram':



→

Constant or 'bias' terms revisited

in statistics the 'all ones' columns are usually included in the 'design matrix'

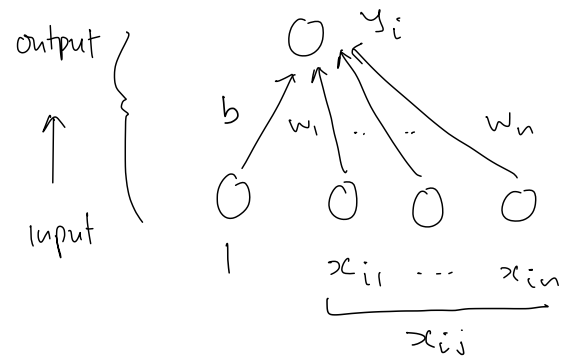
in neural networks the offset or 'bias' terms are often kept separate so we write eg

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} [w] + \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} [b]$$

$$\text{ie } y = Xw + 1b$$

→

More generally, write



for x_{ij} = i^{th} obs. of j^{th} feature, $j=1, \dots, n$

& $X = [x_{ij}]$ matrix

$$\Rightarrow \underbrace{y = Xw + 1b}_{\substack{m \quad (m \times n) \cdot (n)}} \quad \begin{matrix} m \times 1 \\ \text{scalar} \end{matrix}$$

→ so far just a linear/affine model

Nonlinearity & element-wise functions

Next consider a nonlinear transformation of the output of the linear part instead:

$$y_i = f(w x_i + b), \text{ for } \underline{\text{nonlinear } f},$$

$$\text{or } y_i = f(\text{row}_i(x)w + b)$$

For vector inputs, $z = \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_m \end{pmatrix}$ say, Julian notation
define value of f on vector as $f.(z)$ where

$$\begin{array}{l} f \text{ for vector input: } f.(z) \\ \left\{ \begin{array}{l} f.(z) := \begin{pmatrix} f(z_1) \\ f(z_2) \\ \vdots \\ f(z_m) \end{pmatrix} \end{array} \right. \quad \begin{array}{l} \text{(similar for} \\ \text{row vectors \&} \\ \text{matrices)} \end{array} \end{array}$$

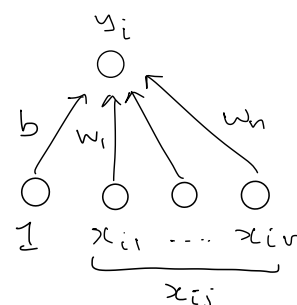
Then

$$y = f.(XW + 1b) \quad \begin{array}{l} \uparrow \\ f \text{ for scalar input as entries} \end{array}$$

Defines a model which is part linear with a subsequent elementwise nonlinear transformation of the output

→ see later for more on element-wise f .

Neuron! (analogy to brain blah blah)



represents:

$$y_i = f(\text{row}_i(x)w + b)$$

$$\text{i.e. } y = f.(XW + 1b) \quad \begin{array}{l} \text{for vector } y: \frac{\text{dim}}{m} \\ \text{matrix } X: m \times n \end{array}$$

w : weights

X : inputs/features

b : bias

f : nonlinear activation function

Note: if f is Heaviside, then 'perceptron' (1958)

if f sigmoid, is 'logistic regression'

can also write $\text{row}_i(x)w$ as $w^T x$ where

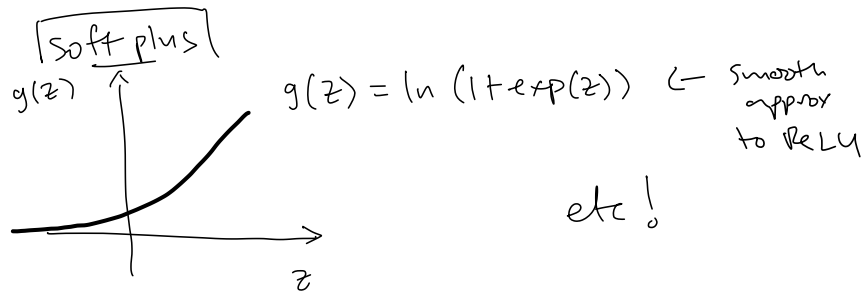
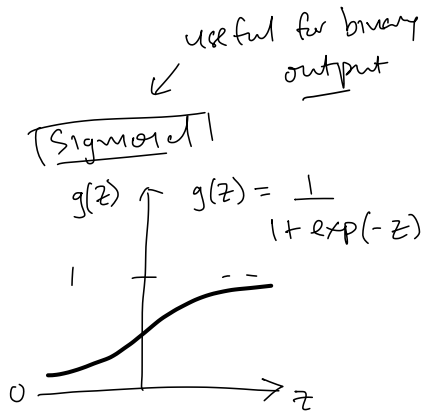
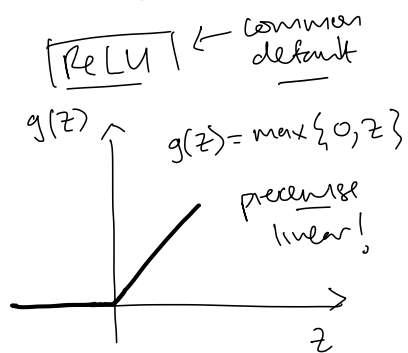
x is $\text{row}_i(x)$ as col. vector
i.e. $[\text{row}_i(x)]^T$

Nonlinearity

If f is linear then it's just a linear model. This is also true for more complex models.

→ f should be nonlinear to be more exciting!

Typical:



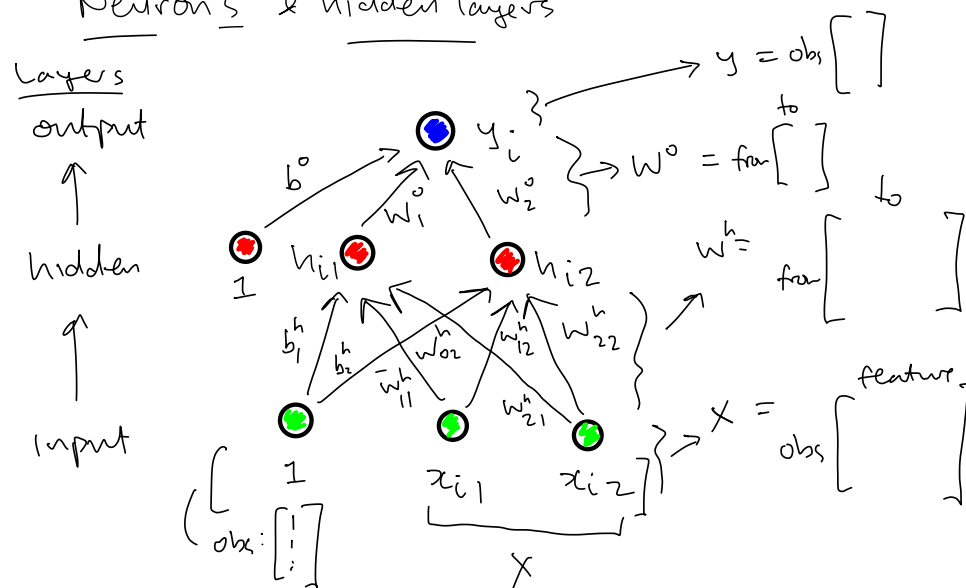
Neurons & hidden layers

A neural network typically contains many, many neurons ('regressions'!)

→ a feedforward neural network (also called multi-layer perceptrons) consists of a series of 'layers', each with multiple neurons:

- each 'layer' has multiple nonlinear (linear) regressions
- the 'next' layer uses the outputs of the previous layer as the inputs/features for its collection of regressions
- layers that aren't observable inputs or outputs are called 'hidden'
- 'deep': many hidden layers.

Neurons & hidden layers



First hidden layer, h :

$$\begin{cases} h_{i1} = f(\text{row}_i(X) w_{[:,1]}^h + b_1^h) \\ h_{i2} = f(\text{row}_i(X) w_{[:,2]}^h + b_2^h) \end{cases}$$

$$\begin{bmatrix} h_{i1} & h_{i2} \end{bmatrix} = f\left(\begin{bmatrix} \text{row}_i(X) & \text{row}_i(X) \end{bmatrix} \begin{bmatrix} w_{[:,1]}^h \\ w_{[:,2]}^h \end{bmatrix} + \begin{bmatrix} b_1^h \\ b_2^h \end{bmatrix} \right)$$

(+ constant for next layer)

Hidden layer cont'd

We've arranged as collection of individual regressions, one per hidden neuron.

Note though:

$$\begin{aligned} \text{vec}(H) &= \text{vec}(f.(XW^h)) \\ &= \text{vec}(f.(XW^hI)) \} \text{vec}(f(\cdot)) \\ &= f.(\text{vec}(XW^hI)) \} f.(\text{vec}(\cdot)) \\ &= f.(I \otimes X) \text{vec}(W^h) \end{aligned}$$

→ could think of as one big regression

e.g. suppose f is identity

then

$$\text{vec}(H) = (I \otimes X) \text{vec}(W^h)$$

$$\begin{bmatrix} h_{i1} \\ h_{i2} \\ \vdots \end{bmatrix} = \begin{bmatrix} X & 0 \\ 0 & X \\ & \ddots \end{bmatrix} \begin{bmatrix} w_{11}^h \\ w_{21}^h \\ \vdots \end{bmatrix}$$

→ stacked regression.

(conceptually, if not computationally)

Output layer

For each observation i , we use the outputs of the hidden layer (here) as the inputs of the output layer

→ do same again! single output here though
→ possibly different activation function, g

→ include constant feature ('bias') from prev. layer $\times$ weight.

$$\begin{aligned} \boxed{y_i = g(H[i,:]W^0 + b)} \quad & W^0: w_{\text{output}} \\ & : P \times 1 \\ & b: \text{scalar bias} \\ \boxed{Y = g(HW^0 + \mathbf{1}b)} \end{aligned}$$

$$\downarrow \begin{bmatrix} \square \\ \square \\ \vdots \end{bmatrix} = g. \left(\begin{bmatrix} \square \\ \square \\ \vdots \end{bmatrix} + \begin{bmatrix} b \end{bmatrix} \right) \left. \vphantom{\begin{bmatrix} \square \\ \square \\ \vdots \end{bmatrix}} \right\} \begin{array}{l} \text{single} \\ \text{'regression'} \\ \text{for single} \\ \text{output node} \end{array}$$

Binary classification: $y \in [0,1]$, class 'probability'

→ use eg logistic, range = $(0,1)$

Element-wise functions

we have considered 'element-wise' functions eg defining for scalar input function $\phi(z)$,

$$\phi.(x) = \begin{bmatrix} \phi(x_1) \\ \phi(x_2) \\ \phi(x_3) \end{bmatrix}$$

for vector input x .

More explicitly we could write

$$\rightarrow \phi.(x) = \begin{bmatrix} \tilde{\phi}_1(x_1, x_2, x_3) \\ \tilde{\phi}_2(x_1, x_2, x_3) \\ \tilde{\phi}_3(x_1, x_2, x_3) \end{bmatrix}$$

$\phi.$ is ϕ for vector input

where $\tilde{\phi}_1(x_1, x_2, x_3) := \phi(x_1)$
 $\tilde{\phi}_2(x_1, x_2, x_3) := \phi(x_2)$
 $\tilde{\phi}_3(x_1, x_2, x_3) := \phi(x_3)$ etc.

ϕ is our original function of a scalar var.

Element-wise functions & derivatives

Note then that the derivative of our elementwise function of a vector is

$$D_x \phi(x) = \frac{\partial \phi(x)}{\partial x^T}$$

$$= \begin{bmatrix} \frac{\partial \tilde{\phi}_1}{\partial x_1} & \frac{\partial \tilde{\phi}_1}{\partial x_2} & \frac{\partial \tilde{\phi}_1}{\partial x_3} \\ \frac{\partial \tilde{\phi}_2}{\partial x_1} & \frac{\partial \tilde{\phi}_2}{\partial x_2} & \frac{\partial \tilde{\phi}_2}{\partial x_3} \\ \frac{\partial \tilde{\phi}_3}{\partial x_1} & \frac{\partial \tilde{\phi}_3}{\partial x_2} & \frac{\partial \tilde{\phi}_3}{\partial x_3} \end{bmatrix}$$

$$= \begin{bmatrix} \phi'(x_1) & 0 & 0 \\ 0 & \phi'(x_2) & 0 \\ 0 & 0 & \phi'(x_3) \end{bmatrix}$$

Scalar
derivative,
i.e.
 $\frac{d\phi}{dx}$ of $\phi(x)$

Define $\phi'(x) =$
↑ vector arg.
scalar deriv

$$\begin{bmatrix} \phi'(x_1) \\ \phi'(x_2) \\ \phi'(x_3) \end{bmatrix}$$

Then $\boxed{D_x \phi(x) = \text{diag}(\phi'(x))}$ $\left\{ \begin{array}{l} \text{diag}(x) \\ = \\ \begin{bmatrix} x_1 & 0 & 0 \\ 0 & x_2 & 0 \\ 0 & 0 & x_3 \end{bmatrix} \\ \text{etc} \end{array} \right\}$

& $\boxed{d\phi(x; dx) = \text{diag}(\phi'(x)) dx}$

Elementwise functions of a matrix

Note: $\text{diag}(x) y = \begin{bmatrix} x_1 y_1 \\ x_2 y_2 \\ x_3 y_3 \end{bmatrix}$ $\left. \begin{array}{l} \text{diag:} \\ \text{scale rows} \\ \text{of } y \text{ by} \\ x \text{ values} \end{array} \right\}$

↑ ↑
vectors

$$= x \odot y, \text{ also!}$$

↑
 $\boxed{\text{Hadamard}}$
(elementwise
product)

matrix
↓

For $D_x \phi(x)$ we could use vec...

but note $y = \phi(x)$

has $\boxed{dy = \phi'(x) \odot dx}$
↑
Hadamard

↳ (can derive via vec form)

Now we are ready to differentiate
neural networks!

→ maybe?

Simple example:

Input $X \in \mathbb{R}^{m \times n}$

Hidden $H \in \mathbb{R}^{n \times p}$

Output $y \in \mathbb{R}^{m \times 1}$

activation ϕ
for hidden
layer

with:

Hidden weights $W^h \in \mathbb{R}^{n \times p}$

Hidden bias $b^h \in \mathbb{R}^p$

Output weights $W^o \in \mathbb{R}^p$

Output bias $b^o \in \mathbb{R}$

→ could just vec & $\otimes$ everything

↳ useful but not usually
approach taken here

↳ let's consider other
approaches..

Detour! Linearityagain

consider $Y(X) = AXB^T$ for matrices
 A, X, B
where A & B
fixed.

This is linear because

$$\begin{aligned} Y(aX_1 + bX_2) &= A(aX_1 + bX_2)B^T \\ &= aAX_1B^T + bAX_2B^T \\ &= aY(X_1) + bY(X_2) \end{aligned}$$

However the 'linear operator' is only
represented implicitly in terms of its
'action' on the 'vector' X (ie the matrix
 X considered as living in a vector/linear space)

write abstractly as

$$\left| \begin{array}{l} Y = L[X] = A \underbrace{X}_{\text{linear operator on } X} \underbrace{B^T}_{\text{action/output}} \end{array} \right|$$

X, 'abstract' not L times X

Explicit via vec & $\otimes$ kron

we have seen we can make an explicit
'vector $\rightarrow$ vector via matrix' representation
using vec & $\otimes$:

$$\text{vec}(Y) = \text{vec}(A X B^T) = (\underbrace{B \otimes A}_{\substack{\text{'big' matrix} \\ \text{matrix as} \\ \text{col. vector}}}) \text{vec}(X)$$

matrix mult

i.e. $\boxed{Y_{\text{vec}} = L X_{\text{vec}}}$

$\rightarrow$ embedding in familiar
col vectors & matrices form

when?

Each has pros & cons!

$\rightarrow$ implicit can be more efficient
& more 'elegant' but explicit
is 'simple' in a verbose way

$\rightarrow$ just usual things are
mapped to column vectors!

There are others too! Tensors & index notation

Differentials & derivatives

eg vec or not
vec

Differentials provide a nice way to generalise
scalar etc calc & are used in most approaches

first: $df = \frac{df}{dx} dx$, $f: \mathbb{R} \rightarrow \mathbb{R}$

now:

$$f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{p \times q}$$

define abstract:

$$\boxed{df = Df [dx]}$$

$\nearrow$ differential output $\uparrow$ linear operator $\nwarrow$ 'differential input'

$\nwarrow$ not matrix mult. $\rightarrow$ fancy D

we can, tho don't always want to, represent

as $\boxed{d(\text{vec } f) = Df d(\text{vec } x)}$

matrix mult.

where Df is the Jacobian matrix
(we called Df in prev. lecture)

Example

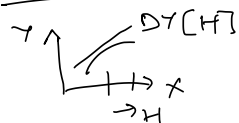
$$Y(x) = A X B^T, \text{ matrices}$$

$$dY = A dX B^T$$

$$\text{defines } DY[dX] = A dX B^T$$

Note: can apply derivative to any input, eg

$$DY[H] = A H B^T$$



→ defined indirectly by action on input

To represent in matrix times vector rep

$$\text{vec}(dY) = (B^T \otimes A) \text{vec}(dX)$$

$$\Rightarrow DY = \underbrace{B^T \otimes A}_{\text{a matrix}}$$

→ explicit matrix rep.

Yet another representation : inner products instead of matrices

Suppose we have a scalar-valued function of a matrix, say } eg loss
 $f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$

Then $Df: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ too

(Riesz rep. thm.)

A famous result is we can always rep. such linear functionals (scalar valued) as the inner product with a 'vector' (in $\mathbb{R}^{m \times n}$):

$$DY[dX] = \langle \nabla_X f, dX \rangle$$

↑ derivative linear operator ↑ gradient vector rep. operator in orig. space $\langle \cdot, \cdot \rangle$ is 'inner product' (like dot prod)

This can be generalised to vector, matrix, tensor etc-valued functions too using 'tensor contractions', ie summing over some but not all vars, given eg matrix → matrix maps etc.

Inner product of matrices & trace

inner product of vectors:

$$\langle x, y \rangle = x^T y$$

Matrices?

$$\hookrightarrow \langle X, Y \rangle = \text{vec}(X)^T \text{vec}(Y) \quad ! \quad \text{vectorised world}$$

Note: result is scalar; multiple ways to compute

can show also have

$$\boxed{\langle X, Y \rangle = \text{tr}(X^T Y)} = \text{vec}(X)^T \text{vec}(Y)$$

$$\text{where } \text{tr}(A) = \text{trace of } A = \sum_i A_{ii} \\ = \text{sum of diagonals}$$

tr has properties:

$$\left\{ \begin{array}{l} \text{tr}(A) = \text{tr}(A^T) \\ \text{tr}(a) = a \quad (\text{scalar } a) \\ \text{tr}(ABC) = \text{tr}(CAB) = \text{tr}(BCA) \end{array} \right.$$

if shapes allow

includes e.g. $A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times m}$

$\hookrightarrow AB$ & BA defined but diff. sizes.

think 'loss'
Derivatives of scalar-valued functions of X

$$df = \langle \nabla_X f, dX \rangle$$

$$= \text{tr}(\nabla_X f^T dX)$$

$\rightarrow$ defines " $\nabla_X f$ " (note: call vec form $\nabla_{\text{vec}} f = \text{vec}(\nabla_X f)$)

$$\text{ex } f(A) = x^T A y, \quad x \in \mathbb{R}^m, A \in \mathbb{R}^{m \times n}, y \in \mathbb{R}^n$$

$$\left. \begin{array}{l} df = x^T dA y \\ = \text{tr}(x^T dA y) \end{array} \right\} \text{scalar } \text{tr}(a) = a$$

$$= \text{tr}(y x^T dA) \quad \left. \right\} \text{cyclic prop.}$$

$$= \text{tr}((y x^T)^T dA)$$

$$\Rightarrow \boxed{\nabla_A f = x y^T} \quad (= x \otimes_T y)$$

$$\text{cf } df = x^T dA y$$

$$\text{vec}(df) = df = \text{vec}(x^T dA y)$$

$$= (y^T \otimes x^T) \text{vec}(dA)$$

$$\Rightarrow Df = y^T \otimes x^T$$

$$\Rightarrow \boxed{\nabla_{\text{vec}} f = y \otimes_k x = \text{vec}(\nabla_A f)}$$

Phew OK! chain rule for differentials

$$\begin{aligned}\text{consider loss } L &= \frac{1}{2} \|y - y_0\|^2 \\ &= \frac{1}{2} (y - y_0)^T (y - y_0) \\ &\quad y \in \mathbb{R}^m\end{aligned}$$

→ easy to show

$$\boxed{\nabla_y L = (y - y_0)}$$

call $E = y - y_0$

Now spec. loss $L = L(y)$, $y = y(u)$ for some matrix U

chain rule:

$$dL(u_0; du) = dL(y; dy(u_0; u))$$

use gradient

$$\begin{aligned}dL &= \langle \nabla_y L, dy(u_0; du) \rangle \\ &= \text{tr}((\nabla_y L)^T dy) \\ &= \text{tr}(E^T dy)\end{aligned}$$

Now ignoring biass & activations for now

$$y = HW^0 \Rightarrow dy = H dW^0 + dH W^0$$

so

$$\begin{aligned}dL &= \text{tr}(E^T dy) \\ &= \text{tr}(E^T H dW^0) + \text{tr}(E^T dH W^0) \quad \left. \begin{array}{l} \text{tr is} \\ \text{linear} \end{array} \right\}\end{aligned}$$

& cyclic property of scalar-valued trace:

$$\text{tr}(E^T dH W^0) = \text{tr}(W^0 E^T dH)$$

$$\text{Now } H = XW^h \Rightarrow dH = X dW^h$$

So

$$\begin{aligned}dL &= \text{tr}(E^T H dW^0) + \text{tr}(W^0 E^T X dW^h) \\ &= \text{tr}((H^T E)^T dW^0) + \text{tr}(X^T E W^0)^T dW^h\end{aligned}$$

$$\Rightarrow \boxed{\begin{aligned}\nabla_{W^0} L &= H^T E \\ \nabla_{W^h} L &= X^T E W^0{}^T\end{aligned}}$$

Including bias

$$y = HW^0 + Ib^0$$

$$H = XW^h + Ib^h$$

↑ treat as
1xP row vector

gives ----

$$\nabla_{W^0} L = H^T E$$

$$\nabla_{W^h} L = X^T E W^{0T}$$

$$\nabla_{b^0} L = I^T E$$

$$\nabla_{b^h} L = I^T E W^{0T}$$

Exercise: verify!

Including activation functions

recall $d\phi(x) = \phi'(x) \circ dX$

↑
Hadamard
elementwise

Write:

$$L = \|y_0 - y\|^2$$

$$y = HW^0 + Ib^0$$

$$H = \underbrace{\phi}_{\text{new}}(XW^h + Ib^h) \quad \left. \begin{array}{l} \text{call } \phi(z) \\ \text{for } z = XW^h + Ib^h \end{array} \right\}$$

$$dy = H dW^0 + dH W^0 + I db^0$$

$$dL = \langle \nabla_y L, dy \rangle$$

$$= \text{tr}((\nabla_y L)^T dy)$$

$$= \text{tr}(E^T dy)$$

$$= \text{tr}(E^T H dW^0) + \text{tr}(W^0 E^T dH) + \text{tr}(E^T I db^0)$$

$$\text{now } dH = \phi'(z) \circ dz, \quad z = XW^h + Ib^h$$

$$\text{so } \text{tr}(W^0 E^T dH) = \text{tr}(W^0 E^T (\phi'(z) \circ dz))$$

→

Elementwise again

can show $\langle A, B \circ C \rangle_F = \langle A \circ B, C \rangle_F$

$$\text{i.e. } \text{tr}(A^T(B \circ C)) = \text{tr}((A \circ B)^T C)$$

$$\Rightarrow \text{tr}(W^0 E^T \phi'(z) \circ dz) = \text{tr}((E W^0)^T \phi'(z) \circ dz)$$

$$= \text{tr}((E W^0 \phi'(z))^T dz)$$

$$dz = x dw^h + 1 db^h$$

$$\rightarrow \text{eg } \nabla_{w^h} L = x^T (E W^0 \phi'(z)) \quad \left. \vphantom{\nabla_{w^h} L} \right\} \text{extra}$$

$$\nabla_{b^h} L = 1^T (E W^0 \phi'(z))$$

$$\text{cf: } \nabla_{w^h} L = x^T E W^{0T}$$

$$\nabla_{b^h} L = 1^T E W^{0T} \quad \left. \vphantom{\nabla_{b^h} L} \right\} \text{before}$$

Phew!

Backpropagation & adjoint/reverse mode diff.

To compute the derivatives via the chain rule we end up needing to compute:

$$ABC \quad \text{or} \quad ABCD$$

for various size arrays (vectors & matrices)

→ while the final answer doesn't depend on the order we multiply these eg $A(BC)$ or $(AB)C$, the computational cost does!

→ in general, want eg vector times matrix instead of matrix times matrix

Note eg

$$\frac{dL}{dw^0} = \underbrace{\text{vector} \times \text{matrix} \times \text{matrix}}_{\text{smaller}} \quad \left. \vphantom{\frac{dL}{dw^0}} \right\} \begin{array}{l} \text{vector} \\ \times \\ (\text{matrix} \\ \times \\ \text{matrix}) \end{array}$$

$$= \underbrace{\{ \text{vector} \times \text{matrix} \}}_{\text{vector} \times \text{matrix}} \times \text{matrix} \quad \left. \vphantom{=}\right\} \begin{array}{l} \text{only vector} \times \text{matrix} \\ \text{ops.} \end{array}$$

⇒ If output is small, use 'backwards order'

→ "backpropagation"!

→ also called adjoint/reverse mode differentiation

Exercises (brave!)

- Implement what we derived (I might later...)
- Compare to eg Pytorch or Jax

↳ does at 'lower' operation level! } computation graphs!
↳ automates for you!

- Read refs on first page (also discuss)

- other refs

- Magnus & Neudecker (1999, 2019)
(Matrix differential calculus)

- Pytorch/Jax documentation

Physics-informed neural nets (PINNs)

→ Combine neural nets + eg differential eqⁿ penalty } (CNN instead of spline)

Scientific Machine Learning Through Physics-Informed Neural Networks: Where we are and What's Next

Salvatore Cuomo¹ · Vincenzo Schiano Di Cola² · Fabio Giampaolo¹ · Gianluigi Rozza³ · Maziar Raissi⁴ · Francesco Piccialli¹

Received: 14 January 2022 / Revised: 6 July 2022 / Accepted: 6 July 2022 /
Published online: 26 July 2022
© The Author(s) 2022

Abstract

Physics-Informed Neural Networks (PINN) are neural networks (NNs) that encode model equations, like Partial Differential Equations (PDE), as a component of the neural network itself. PINNs are nowadays used to solve PDEs, fractional equations, integral-differential equations, and stochastic PDEs. This novel methodology has arisen as a multi-task learning framework in which a NN must fit observed data while reducing a PDE residual. This article provides a comprehensive review of the literature on PINNs: while the primary goal of the study was to characterize these networks and their related advantages and disadvantages. The review also attempts to incorporate publications on a broader range of collocation-based physics informed neural networks, which stars form the vanilla PINN, as well as many other variants, such as physics-constrained neural networks (PCNN), variational hp-VPINN, and conservative PINN (CPINN). The study indicates that most research has focused on customizing the PINN through different activation functions, gradient optimization techniques, neural network structures, and loss function structures. Despite the wide range of applications for which PINNs have been used, by demonstrating their ability to be more feasible in some contexts than classical numerical techniques like Finite Element Method (FEM), advancements are still possible, most notably theoretical issues that remain unresolved.

Keywords Physics-Informed Neural Networks · Scientific Machine Learning · Deep Neural Networks · Nonlinear equations · Numerical methods · Partial Differential Equations · Uncertainty

Physics-informed neural networks (PINNs) for fluid mechanics: a review

Shengze Cai¹ · Zhiping Mao² · Zhicheng Wang³ · Minglang Yin^{4,5} · George Em Karniadakis^{1,4}

Received: 21 May 2021 / Accepted: 10 September 2021 / Published online: 23 January 2022
© The Chinese Society of Theoretical and Applied Mechanics and Springer-Verlag GmbH Germany, part of Springer Nature 2021

Abstract

Despite the significant progress over the last 50 years in simulating flow problems using numerical discretization of the Navier-Stokes equations (NSE), we still cannot incorporate seamlessly noisy data into existing algorithms, mesh-generation is complex, and we cannot tackle high-dimensional problems governed by parametrized NSE. Moreover, solving *inverse flow problems* is often prohibitively expensive and requires complex and expensive formulations and new computer codes. Here, we review *flow physics-informed learning*, integrating seamlessly data and mathematical models, and implement them using physics-informed neural networks (PINNs). We demonstrate the effectiveness of PINNs for inverse problems related to three-dimensional wake flows, supersonic flows, and biomedical flows.

Keywords Physics-informed learning · PINNs · Inverse problems · Supersonic flows · Biomedical flows